

Product Requirements Document

Book Catalog API — FastAPI + SQLAlchemy ORM + UI

STUDENT ASSESSMENT

Document Type	Hands-on Build Assessment (PRD Brief)	Duration	1–2 hours
Tech Focus	FastAPI, SQLAlchemy ORM, SQLite, Basic UI	Mode	Individual

1. Overview

Build a small full-stack **Book Catalog** application. The backend exposes a REST API built with **FastAPI**, using **SQLAlchemy ORM** to persist data to a database. A simple UI connects to the API so a user can add, view, update, and delete books from the browser.

The goal is to demonstrate that the API and database layer are correctly wired together, and that the UI can perform full CRUD operations end-to-end against the live API.

2. Objective

Given a book with fields such as title, author, and status, the candidate must design the SQLAlchemy model, expose CRUD endpoints via FastAPI, and build a minimal UI that consumes those endpoints — proving a working connection across all three layers (UI → API → Database).

3. Scope

In Scope

- One resource: `Book`
- Full CRUD via FastAPI endpoints
- SQLAlchemy ORM model + SQLite database
- Pydantic request/response schemas
- Minimal UI (HTML/JS, or any framework) to call the API

Out of Scope

- User authentication / login
- Multiple resources or relationships
- Deployment / hosting
- Styling polish or responsive design

4. Tech Stack

Layer	Technology
Backend Framework	FastAPI
ORM	SQLAlchemy

Database	SQLite (default) — any SQL DB accepted
Validation	Pydantic
Server	Uvicorn
Frontend	Plain HTML/CSS/JS (fetch/axios) — any framework is acceptable

5. Data Model — Book

Field	Type	Notes
id	Integer	Primary key, auto-increment
title	String	Required
author	String	Required
genre	String	Optional
status	String / Enum	available · borrowed (default: available)
added_at	DateTime	Auto-set on creation

6. Functional Requirements

ID	Requirement
FR-1	API can add a new book (title & author required, genre & status optional)
FR-2	API can list all books and fetch a single book by ID
FR-3	API can update an existing book's details, including status
FR-4	API can delete a book by ID
FR-5	UI connects to the API and lets a user add, view, update, and delete books without using Swagger/Postman

7. API Endpoints (Suggested)

Method	Endpoint	Description
POST	/books	Add a book
GET	/books	List all books
GET	/books/{id}	Get a single book
PUT	/books/{id}	Update a book
DELETE	/books/{id}	Delete a book

8. UI Requirements

- A form to add a new book (title, author, genre, status)
- A list/table view showing all books, pulled live from the API
- Ability to mark a book's status or edit it, and delete it, from the UI
- No design system required — functionality over aesthetics

9. Non-Functional Requirements

- Code should run locally with a single command (e.g. `uvicorn main:app --reload`)
- Basic error handling (e.g. 404 for a book ID that doesn't exist)
- Clean, readable project structure (see below)

10. Suggested Project Structure

```
project/  
  main.py    # FastAPI app & routes  
  models.py  # SQLAlchemy models  
  schemas.py # Pydantic schemas  
  database.py # DB engine / session setup  
  static/ or templates/ # UI files  
  requirements.txt
```

11. Deliverables

- Source code (backend + UI)
- `requirements.txt` with dependencies used
- Brief note (2–3 lines) on how to run the app locally

12. Evaluation Criteria

Criteria	Weight
API + ORM correctly connected to the database	35%
All CRUD operations working end-to-end	30%
UI successfully calls and reflects API/DB data	25%
Code clarity & structure	10%